

RewardSense User Platform Expansion Plan

Summary

Extend the current React + FastAPI demo into a lightweight user product on top of the existing recommendation stack. The implementation should keep the current scoring/personalization/LLM serving flow, but add an application layer with auth, persistent user data, richer recommendation inputs, transaction logging, savings views, feedback capture, and business-facing LLM/latency/cost reporting.

Chosen defaults for this plan:

- Keep the existing FastAPI backend and add a lightweight SQLite relational store
- Deliver as a phased roadmap organized by epics and implementation stories
- Use email/password auth only
- Use preset selectable personas for recommendation shaping
- Store user feedback for analytics and future tuning, not immediate reranking
- Make transaction logging opt-in; if disabled, recommendations remain available but more generic
- Use stable API/view-model contracts as the “UI agnostic” strategy
- Use static curated card image assets first
- Provide CSV/XLSX export first; no Google Sheets integration in this roadmap
- Provide on-demand HTML/PDF business reports for latency/cost monitoring

Public Interfaces And Data Model Changes

New API surface

Add an application API layer alongside the existing prediction/monitoring endpoints.

- `POST /auth/signup`
 - create account with email, password, display name
 - reject duplicate email
- `POST /auth/login`
 - validate existing signed-up user only
 - return auth token + current user profile summary
- `POST /auth/logout`
- `GET /me`
 - current user, personas, logging preference, saved cards, profile settings
- `PATCH /me/profile`

- update display fields, personas, reward preferences, dark mode preference, transaction logging opt-in
- `GET /cards/catalog`
 - stable UI-facing card list with image URL, issuer, fee, reward highlights
- `PUT /me/cards`
 - replace saved-card portfolio for the user
- `POST /recommendations/portfolio`
 - recommendation using saved profile + saved cards + broad spending inputs
- `POST /recommendations/transaction`
 - single-merchant/single-transaction recommendation using typed merchant, amount, category hints, and saved cards
- `POST /transactions`
 - add a user transaction log entry if logging opt-in is enabled
- `GET /transactions`
 - paginated transaction history
- `GET /transactions/export?format=csv|xlsx`
- `POST /feedback`
 - like/dislike + optional reason against a recommendation or card suggestion
- `GET /summary`
 - aggregate spend/savings totals for charts and category breakdowns
- `GET /reports/business-metrics`
 - trigger or fetch latest HTML/PDF latency/cost report metadata

New persistent entities

Add relational tables for:

- users
- auth_credentials
- user_settings
- user_personas
- saved_cards
- recommendation_events
- transaction_logs
- feedback_events
- llm_telemetry
- business_metric_reports

Stable UI-facing contracts

Do not expose raw model internals directly to pages.

Create backend response shapes that normalize:

- card display fields
- score display fields
- savings impact fields
- explanation fields

- chart-ready summary data
- profile/settings data

Implementation Plan

Epic 1: Account Foundation And User Persistence

Story 1.1: Auth and protected app shell

- Add signup/login/logout flow with email/password only
- Hash passwords with a standard password hasher and store no plaintext credentials
- Use signed JWT access tokens for the first version; require re-login on expiry rather than adding refresh-token complexity
- Add frontend auth context, route guards, and redirect unauthenticated users away from dashboard/profile/history pages
- Update the landing page CTA flow so logged-out users are routed through auth before protected features

Story 1.2: User profile and settings model

- Add editable user profile with display name, selected personas, reward preference, transaction logging opt-in, and dark mode preference
- Seed persona definitions in backend config so behavior is deterministic and explainable
- Store portfolio cards separately from recommendation requests so users can build a persistent wallet

Story 1.3: Persona-aware recommendation inputs

- Represent personas as explicit recommendation modifiers, not inferred-only hidden state
- Initial persona modifiers:
 - student: penalize annual fees strongly, favor no-fee cards and simple cashback
 - traveler: boost travel multipliers, lounge/travel benefit cards, transferable points
 - family: boost groceries/gas/utilities and lower fee sensitivity than student
 - cashback-focused: favor simple flat-rate and cashback cards
- Apply persona adjustments in the application scoring layer before final ranking so the behavior is transparent and easy to demo
- Include persona context in the explanation payload

Epic 2: Recommendation Experience Expansion

Story 2.1: Portfolio-based recommendations

- Rework the current recommendation form to read from the authenticated user profile by default
- Use saved cards as the default candidate wallet for recommendations
- If the user has no saved cards, fall back to the curated catalog and clearly label results as generic
- Make recommendation responses include:

- top card
- ranked alternatives
- score breakdown
- why it matches the selected persona(s)
- projected savings

Story 2.2: Single-transaction recommendation page

- Add a dedicated page for quick transaction entry like “McDonald’s, \$15”
- Accept merchant name, amount, optional category, and optional date
- Resolve category using simple merchant/category heuristics first; if unresolved, fall back to user-chosen category
- Rank only the user’s saved cards for this flow unless the user explicitly asks for “best new card” behavior later
- Return a clear recommendation, estimated reward, and “money saved” value for that purchase

Story 2.3: Savings calculator and card impact views

- Add a calculator showing estimated savings by category and total monthly/annual savings for each recommended card
- Use the user’s spending profile or typed transaction amount as the input basis
- Baseline for “savings”:
 - compare against the user’s first saved catch-all card if one exists
 - otherwise compare against a generic 1% cashback baseline
- Show category-by-category uplift in a chart/table so the demo makes the recommendation logic visible

Story 2.4: Card images and display contracts

- Add static curated card image assets keyed by stable card IDs/slugs
- Return image URLs from the catalog and recommendation payloads so the frontend never guesses image paths
- Normalize card presentation fields into one UI-ready shape used by home, result, wallet, and calculator pages

Epic 3: Transaction Ledger, Summary, And Export

Story 3.1: Opt-in transaction logging

- Add a clear user setting for transaction logging
- Explain in UI copy that disabling history will make recommendations more generic because the system loses behavioral context
- Only persist personal transaction history if opt-in is enabled
- Allow manual transaction entry from the single-transaction page and optional recommendation-result logging from broader recommendation flows

Story 3.2: Logged transaction schema

- Store merchant, amount, category, chosen card, reward earned, estimated savings vs baseline, timestamp, and source flow
- Store whether the card was user-owned/saved at the time of logging
- Link transactions to recommendation events when applicable so feedback and explanation telemetry can later be analyzed together

Story 3.3: Summary page

- Add a Splitwise-style summary page with chart-ready aggregates:
 - spend by category
 - rewards earned by category
 - savings by card
 - fee-adjusted savings totals
- Use pie/bar charts and a short “top insights” section for the demo
- If logging is disabled or the user has no data, show an empty state that points them back to the transaction page

Story 3.4: Export

- Add CSV and XLSX download support from transaction history
- Export only user-owned rows and include all summary-relevant fields
- No direct Google Sheets sync in this roadmap

Epic 4: Feedback, LLM Quality, And Telemetry

Story 4.1: Feedback capture

- Add like/dislike actions on recommendation cards and explanations
- Allow optional short feedback reason tags such as “too expensive”, “not relevant”, “already have this card”, “explanation unclear”
- Persist feedback with user, card, recommendation event, and timestamp
- Do not change live ranking in v1; expose this data for later retraining and analytics

Story 4.2: Better explanations

- Update explanation prompting/output contract so each response includes:
 - a specific summary
 - exactly 2 pros
 - exactly 2 cons
 - a short “best for” line when appropriate
- Ensure prompts include persona context, fee sensitivity, saved-card context, and the score breakdown behind the recommendation
- Strengthen output validation so low-quality responses fall back to deterministic templates with the same 2-pro / 2-con structure

Story 4.3: LLM telemetry and prompt drift tracking

- Persist per-explanation telemetry:

- prompt version/hash
- model name
- temperature
- latency
- fallback usage
- token/cost estimate when derivable
- Add "prompt drift" as a change-tracking concept based on prompt version/hash and output-quality shifts across time
- Keep this logging separate from user-facing history so the UI remains implementation-agnostic

Epic 5: Business Monitoring And Reporting

Story 5.1: Business metrics collection

- Reuse existing inference logging and extend it with:
 - request counts
 - stage latencies
 - LLM call counts
 - estimated token/cost metrics
 - success/fallback/error rates
- Aggregate these by day and report window rather than exposing raw records to the demo UI

Story 5.2: On-demand report generator

- Add a script/job that compiles latency/cost/business usage metrics into HTML and PDF
- Report contents:
 - total requests
 - average/p95 latency
 - explanation latency split
 - estimated LLM cost by period
 - fallback/error rates
 - top-used recommendation flows
- Keep this outside the user product UI; it is an internal/business artifact

Epic 6: Frontend Refresh And Design System Hardening

Story 6.1: App shell and route expansion

- Add pages for:
 - login/signup
 - profile/settings
 - wallet/saved cards
 - quick transaction recommendation
 - transaction history
 - expense summary
- Keep all pages consuming typed API view models from a single frontend service layer

Story 6.2: Dark mode update

- Change dark mode to true black background with accent color highlights
- Preserve accessibility and contrast for charts, cards, and forms
- Keep the theme toggle but persist preference in the user settings profile

Story 6.3: UI abstraction boundary

- Standardize frontend data access behind typed API client functions and page-level view models
- Avoid leaking backend/model field names directly into presentation components
- Keep the UI resilient if recommendation internals change later, so long as the public API contracts stay stable

Test Plan

- Auth:
 - signup succeeds for new user
 - duplicate signup fails
 - login only works for signed-up users with correct password
 - protected routes reject unauthenticated access
- Profile/personas:
 - persona updates persist
 - student persona lowers ranking of high-fee cards relative to neutral profile
 - generic recommendations are returned when transaction logging is disabled or history is absent
- Saved cards:
 - wallet CRUD works
 - single-transaction recommendation only uses saved cards when wallet exists
- Recommendations:
 - portfolio recommendation returns stable UI-facing payload
 - quick transaction flow returns card, reward, and estimated savings
 - savings calculator uses saved-card baseline when available and 1% baseline otherwise
- Transaction ledger:
 - opt-in required before persistence
 - manual log entry stores merchant/card/savings fields correctly
 - summary endpoint aggregates category totals correctly
 - CSV/XLSX export schema matches stored fields
- Feedback:
 - like/dislike events persist with recommendation linkage
 - feedback storage does not alter immediate ranking
- LLM:
 - responses produce exactly 2 pros and 2 cons when successful
 - bad/timeout responses fall back cleanly

- telemetry captures prompt version/hash, temperature, latency, and fallback status
- Monitoring/reporting:
 - business report generator produces HTML and PDF outputs
 - latency/cost aggregation handles empty periods and partial LLM usage
- Frontend:
 - protected navigation flow works
 - dark mode renders with black background and preserved contrast
 - card images load from curated assets for supported cards

Assumptions And Defaults

- The current FastAPI app remains the primary backend; no separate app server is introduced
- SQLite is acceptable for this phase and the schema is designed so it can later migrate to Postgres with minimal API changes
- Email/password auth is sufficient for the current roadmap; no social auth or passwordless flow
- Recommendation quality changes in this phase are application-layer integrations around the existing scoring/personalization pipeline, not a redesign of the training DAGs
- Feedback is collected now for analytics and future ranking/model work, not immediate closed-loop personalization
- Google Sheets sync is out of scope for this roadmap; export is download-first
- “UI agnostic” is implemented through stable API contracts and frontend view models, not a deeper cross-framework abstraction effort

